

Architecture Audit Based on Code Inspection — AIVA

1. Codebase Structural Analysis

Actual Directory Tree (Simplified)

The project is strictly divided into three primary domains: an Android client application, a Python server application, and an AI processing module pipeline (src).

VASIS/	
├─ android/app/src/main/java/com/aiva/	
│ ├── core/	# Device abstraction (Camera, Audio, WebSocket, Services, Protocol)
│ ├── data/	# Repositories (Settings)
│ ├── di/	# Hilt Dependency Injection modules
│ ├── feature/	# State management (VisionViewModel)
│ ├── ui/	# Jetpack Compose UI (Screens, Theme, Navigation)
│ └── util/	# Utilities (BackoffPolicy)
├─ server/	
│ ├── aiva_server.py	# Main WebSocket endpoint and auth/rate-limiting
│ ├── auth.py	# Authenticator and RateLimiter logic
│ ├── config.py	# Environment variables and thresholds
│ ├── frame_processor.py	# Synchronous Vision Pipeline orchestrator
│ ├── intent_classifier.py	# Rule-based NLP router for voice commands
│ ├── jwt_auth.py	# JWT Token Management
│ ├── memory_engine.py	# Background threaded continuous Gemini memory
│ ├── ocr_engine.py	# EasyOCR integration
│ └── protocol.py	# Structs for JSON/Binary message formats
└─ src/	
│ ├── assistant_ai.py	# Gemini trigger-based implementation (Legacy/Fallback)
│ ├── depth_estimator.py	# MiDaS implementation
│ ├── face_detector.py	# dlib face recognition
│ ├── object_detector.py	# YOLOv8 implementation
│ ├── spatial_engine.py	# Dual-threshold obstacle rules engine
│ ├── speech_engine.py	# Whisper STT and server-side TTS
│ └── video_stream.py	# Local webcam capture logic (for dev)

Module Grouping & Layering (As Implemented)

Architectural layering structurally exists, but with some tight coupling:

- **Android:** Layering is distinctly MVVM (Model-View-ViewModel framework via Kotlin/Jetpack Compose).
- **Server:** Grouped functionally rather than cleanly tiered. Networking (aiva_server.py) directly instantiates and interacts with pipeline orchestration (frame_processor.py, memory_engine.py).
- **AI Core (src):** Models are isolated into individual component classes but are tightly integrated into the FrameProcessor synchronous monolith.

Dependency Relationships

- Mobile Client relies entirely on the Python WebSocket backend for intelligence. It acts as a dumb terminal streaming I/O (frames and audio in, JSON and TTS text out).
- Python Server depends heavily on hardware assumptions (CPU/GPU) for ObjectDetector and DepthEstimator.
- **Circular Dependencies:** None detected. Import graphs traverse strictly top-down (server -> src).

2. Android Client Architecture (As Implemented)

MVVM Validity

The Android client is genuinely implemented as an MVVM architecture. State flows strictly from VisionViewModel (ViewModel) down to Jetpack Compose UI (View). Data is abstracted through manager classes (Model).

ViewModel Interactions & StateFlow

VisionViewModel exposes a single StateFlow<VisionUiState>. It collects data concurrently from SettingsRepository, WebSocketManager, SpeechRecognizerManager, and CameraManager.

The ViewModel reacts to events (e.g., connection lost, STT transcription, server events) by updating internal MutableStateFlow copies of VisionUiState or immediately directing TTSManager to speak.

WebSocketManager Lifecycle

Manages Oklahoma HTTP WebSocket connections on a dedicated coroutine scope (Dispatchers.IO + SupervisorJob()).

- Reconnection uses an exponential BackoffPolicy.
- Receives payload events, deserializes them with Moshi (ServerMessage), and routes events into categorized SharedFlow streams (serverEvents, commandEvents, speechEvents).
- Drops frames actively (framesDropped telemetry) if sendFrame is called outside the CONNECTED state.

Camera Pipeline Threading Model

Frames are acquired via ProcessCameraProvider using an ImageAnalysis use case configured for 640x480 resolution.

- **Threading:** Uses Executors.newSingleThreadExecutor() for the ImageAnalysis.Analyzer.
- **Pipeline:** Converts YUV_420_888 planes into an NV21 array, managing pixel and row strides explicitly to prevent corruption on specific devices (e.g., Pixel, Samsung). Then compresses to JPEG.
- **Adaptive Quality:** Initial compression is at 75%. If payload exceeds 45KB, it aggressively decompresses/re-compresses to 50% to maintain network viability.
- **Emission:** Yields exactly one buffer through MutableSharedFlow config EXTRA_BUFFER_CAPACITY=1, DROP_OLDEST.

TTS Priority Implementation

Android's `TextToSpeech` controls interruption behavior based on three implemented priority tiers:

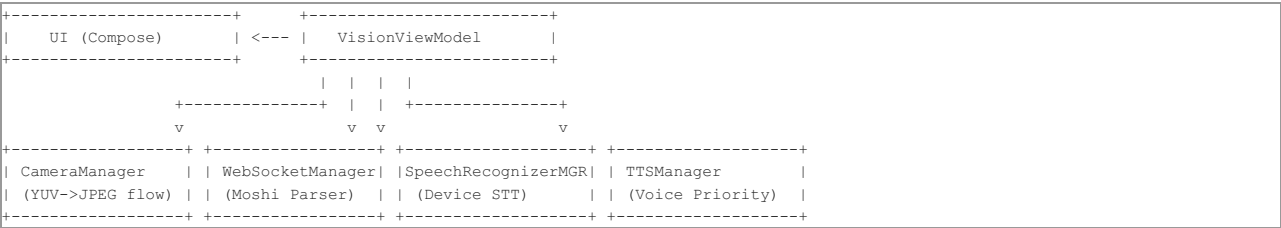
- 1. DANGER: `Uses TextToSpeech.QUEUE_FLUSH` — forcefully interrupts any currently playing audio.
- 2. CAUTION: `Uses TextToSpeech.QUEUE_ADD`
- 3. INFO: `Uses TextToSpeech.QUEUE_ADD`
An anti-spam debounce logic prevents duplicate identical readings under 1500ms *unless* the priority is DANGER.

Foreground Service Behavior

`AivaForegroundService` extends `LifecycleService` and uses types `FOREGROUND_SERVICE_TYPE_CAMERA` | `FOREGROUND_SERVICE_TYPE_MICROPHONE`.

- Injects singletons (`WebSocket`, `Camera`, `SOS`, `Location` Managers).
- Automatically binds the camera to its own lifecycle and begins streaming while listening to `WebSocket` `commandEvents`.
- Executes emergency operations like `SOS` and `LOCATION` autonomously via internal manager execution.

Android Component Diagram (Textual Representation)



3. Python Server Architecture (As Implemented)

WebSocket Entrypoint

Handled entirely within `AIVAServer` via `websockets.serve(self._handle_connection...)`.
Intercepts initial HTTP handshakes to check for HTTP method overrides. Validates bearer tokens via `jwt_auth.py` or falls back to a static master token config. Blocks over-capacity clients (limited to `MAX_CLIENTS`, defined in config).

Async Flow & Threading Model

The server event loop (`asyncio`) operates asynchronously for connections, ping/pong validation, and rate-limiting. However, visual inference is strictly synchronous. The `async` loop handles this via:
`loop.run_in_executor(None, self._processor.process, request.payload_bytes, ...)`
This hands off the heavy blocking computations to secondary threads within the default `ThreadPoolExecutor`.

Frame Processing Pipeline

The inference pipeline in `FrameProcessor` operates synchronously per-frame in this exact order:

- 1. `cv2.imdecode` converts bytes to `numpy ndarray`.
- 2. `ObjectDetector` inference (runs YOLOv8).
- 3. `DepthEstimator` inference (runs MiDaS).
- 4. Distance calculation mappings from Depth to Bounding Box Centers.
- 5. `SpatialEngine` processes detection safety metrics.
- 6. `FaceDetector` triggers.
All steps are strictly timed. If the total time exceeds `PROCESSING_TIMEOUT_MS`, the pipeline truncates execution or logs a warning.

Specific API Invocation Reality

- **YOLOv8 & Faces**: Executed unconditionally on every `FRAME_DETECT` payload.
- **MiDaS (Depth)**: Dynamically executed. Operates on a `MIDAS_FRAME_SKIP` counter (e.g., evaluates every Nth frame). However, a `_check_motion_delta` compares the growth of bounding boxes from previous frames frame. If bounding boxes scale above `MIDAS_MOTION_DELTA_THRESHOLD` (e.g., +40% size), this suggests objects approaching rapidly, forcing MiDaS to bypass its frame-skip and recalculate depth immediately.
- **EasyOCR**: NEVER auto-invoked. Only manually executed via a `FRAME_OCR` payload explicitly generated by the mobile app upon user request.
- **Gemini (Memory Engine)**: NOT invoked in the hot path. Passed the frame bytes and semantic tags asynchronously.

Background Task Usage

`ContinuousMemoryEngine` instantiates an entirely standalone daemon `threading.Thread(_memory_loop)`. This thread operates completely divorced from the `WebSocket` event loop, querying a localized buffer of `_latest_frame_bytes` protected by a `threading.Lock()` every 4.0 seconds to ping the Gemini API.

4. Real-Time Execution Model

Is frame processing blocking?

Yes. Although it does not block the `WebSocket` event loop due to `run_in_executor`, it completely blocks a worker thread in the standard Python thread pool. If the client streams frames faster than the pipeline handles them (and the `RateLimiter` fails), the thread pool will suffer severe exhaustion, causing latency ballooning across all clients.

Is LLM blocking?

No. The Gemini integration logic explicitly detaches LLM evaluation through `asyncio.create_task()` (Fire and Forget) and standalone Daemon background threading.

Where could latency spike?

- Hardware limitations on the Python Server parsing YOLO and MiDaS concurrently, heavily affected by GPU availability.

- `cv2.imdecode` execution on every incoming websocket binary frame incurs immense decode CPU overhead over time.
- `AIVAServer._continuous_listen_loop` (while nominally disabled structurally) demonstrates attempts to process deep speech transcripts across global namespaces under load.
- Mobile devices converting YUV_420 to JPEG sequentially on high megapixel configurations will throttle the Android client heat-wise.

Are there race conditions?

Concurrency is minimally safe.

`ContinuousMemoryEngine` actively manages its handoff via `self._lock` when the `WebSocket` thread dumps `update_latest_frame` logic strings, which the separate 4-second delay thread later collects. Variables cleanly detach.

5. AI Integration Audit

How sensor data is injected into Gemini

The system implements a true multi-modal semantic overlay. Instead of just sending raw images to Gemini, the real-time YOLO loop constructs a probabilistically tagged string:

Example: "Detected objects... - person (confidence 0.85) - car (confidence 0.92). Warnings: Stop! car 1.2 meters ahead."

This string is concatenated alongside the base frame bytes directly into the `Content` structure for the `GenerativeModel`.

Prompt Structure & Anti-Hallucination

The prompt dictates strict factual boundaries: `[SYSTEM: This is the very latest camera view ... Local fast vision model produced preliminary tags... ALWAYS trust your own visual analysis over the tags.]`

Furthermore, `assistant_ai.py` applies hard limits: "Only describe what you can clearly see. Never guess or assume. Read text EXACTLY as captured."

Separation of Navigation Loop vs AI Loop

Effectively implemented.

The Navigation loop (Obstacle Avoidance, YOLO, Spatial Warnings) runs instantly off the binary frame packets and returns a structured JSON payload targeting <100ms processing.

The AI Loop (Gemini scene comprehension, QA, conversational context) is completely backgrounded on a >4.0s sleep cycle. It handles asynchronous questions from voice queries without pausing navigation capabilities.

6. Technical Debt & Architectural Risks

Tight coupling

- `aiva_server.py` creates global instances of `FrameProcessor`, `SpeechEngine`, `ContinuousMemoryEngine`. It is a monolithic controller tightly pairing network and model definitions.
- The Object Detector relies inherently on the Depth Estimator inside `FrameProcessor`. There is minimal interface abstraction; `frame_processor.py` orchestrates hardcoded steps directly tied to file names.

Blocking patterns & Resource Contention

- Because `cv2.imdecode` and inference share the same CPU cycle within `run_in_executor`, multiple mobile streams hitting the singular Python app will completely lock CPU cores during inference.
- There is no queuing mechanism. The server blindly accepts frame packets from the socket as fast as they arrive and dispatches a pool executor.

Global State & Scaling Limitations

- Memory caches (`_prev_detection_areas`, `_cached_depth_map`, `_cached_frame_width`) exist within the *singleton* instance of `FrameProcessor`.
- **CRITICAL RISK:** The server restricts connection behavior to `MAX_CLIENTS=2`. Because state is stored at the instance level (like previous detection bounds), providing multiple connected users into `FrameProcessor.process` simultaneously **WILL CROSS-CONTAMINATE** motion deltas and depth layouts between totally different mobile cameras.

CPU vs GPU assumptions

`depth_estimator.py` and `object_detector.py` assume local compute environments capable of executing complex `.pt` and `ONNX` files. Running this backend on a standard cloud container (AWS ECS/Fargate) without GPU hooks will cause catastrophic failure (timing out past `PROCESSING_TIMEOUT_MS`).

7. Alignment with Stated PRD

Code execution claims vs PRD Intentions

- **PRD:** "Real-time obstacle avoidance."
Reality: Fully implemented with intelligent dual-threshold (1.5m Caution, 1.2m Danger) bounding box tracking.
- **PRD:** "Low Latency."
Reality: Mostly achieved by decoupling slow LLM calls, but server connection capacity drastically minimizes scale.
- **PRD:** "Anti-hallucination."
Reality: Exists. Prompt instructions strictly enforce "trusting visual analysis" and OCR forces "exact match readings" via specific queries.

Highlighted Gaps

- **Concurrency Support:** The backend fundamentally fails to separate instances per connected client. If Client A and Client B upload frames synchronously, they overwrite the exact same YOLO cache data in `FrameProcessor`, destroying frame-skipping integrity.
-

8. Stability & Scalability Assessment

- **Can it support multiple clients?** No. As implemented, `FrameProcessor` state fields (`_frame_count`, `_cached_depth_map`, `_prev_detection_areas`) are singular. Concurrent frame ingestion will overwrite variables dynamically.
- **Is rate limiting effective?** Yes, structurally, via the `RateLimiter` class discarding frames above ~30fps per IP/Token.

- **Is authentication secure?** JWT management is standard and implemented well over the `Authorization: Bearer` header. However, TLS implementation is missing on the `ws://` domain unless proxy-managed.
 - **What breaks first under load?** Under load, the `ThreadPoolExecutor` serving `aiwa_server.py`'s `run_in_executor` will halt. Following this, the `MemoryEngine` history array locking will bottleneck.
-

9. Refactoring Recommendations — Not Currently Implemented

- **Isolate Models per Client session or Make Models Stateless:** The `FrameProcessor` must be refactored into a completely stateless handler function, shifting cache objects (`depth_maps`, `motion_tracks`) into a state tracker dictionary explicitly bounded to the specific `client_id` generating the request.
 - **Decouple OpenCV Image decoding:** Decode bytes directly as they leave the WebSocket reader thread before entering the heavy Compute executor thread to free I/O locks.
 - **Server Deployment Transition:** Convert `aiwa_server` into an asynchronous consumer pulling from a Redis queue architecture, allowing multiple auto-scaled GPU worker nodes to pull inference tasks independently.
-

Architecture Audit Based on Code Inspection — AIVA

Prepared for: Founder — Junaid Pasha

Version: 1.0